

TallyCheck

Technical overview and build guide

How the application is made, how its parts fit together, and how the Windows installer is built, signed and updated.

Version	1.0.0
Date	2 November 2026
Publisher	Cenango (Pvt) Ltd
License	MIT OR Apache-2.0 (names and logos excluded, see NOTICE)
Source code	https://github.com/Sammysam/tallycheck
Project folder	C:\dev\accounting-desktop

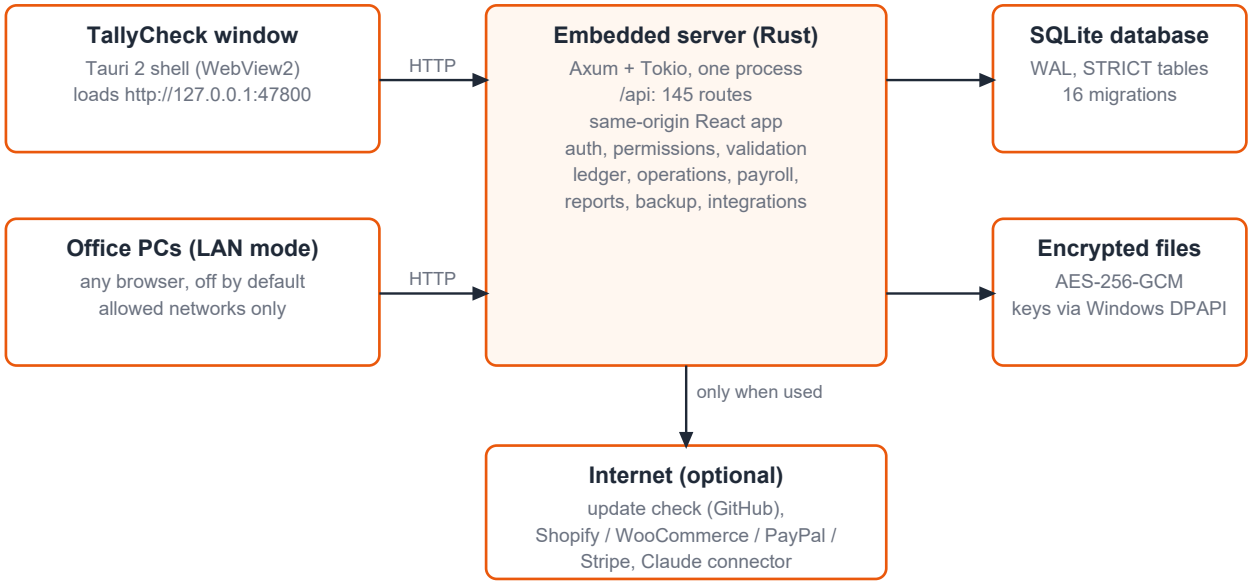
1. What TallyCheck is

TallyCheck is a fully offline accounting, invoicing, payroll and inventory application for Windows. It is a desktop port of the Cenango accounting web application (NestJS + React + PostgreSQL), rebuilt as a single Windows program: a Rust server with an SQLite database, and the original React screens shown in a native window. No Node.js, PostgreSQL, Docker or cloud service is needed at run time.

Fact	Value
Rust code	38,146 lines in 131 files, 15 crates + the desktop shell
Frontend code	16,626 lines (React 19, TypeScript), about 100 files
API	145 routes under /api, same paths and JSON as the original Nest API
Database	SQLite, 16 versioned migrations (1,039 lines of SQL)
Automated tests	235 Rust tests (unit + API), contract tests against the original, 54-step UI smoke test
History	48 commits; 48 recorded design decisions (D1–D48)
Installer	NSIS, per-user, about 11 MB, no administrator rights needed

2. Architecture

Everything runs in **one process**. The Tauri shell starts the embedded Axum server on 127.0.0.1, then opens a window on that address. The server serves both the React app and /api from the same origin, so the frontend's `baseURL: '/api'` works unchanged and no CORS layer is needed. Other office PCs (LAN mode, off by default) use a browser against the main PC's server; they never open the database file.



The window has no native access (no Tauri IPC is granted to the page), so the frontend behaves exactly as in a browser. Features that need the operating system — the in-app updater — run in the Rust shell and reach the page only through the server's own API.

3. Technology stack

Layer	Technology	Why
Desktop shell	Tauri 2 (WebView2), Rust	Small native window, no bundled browser engine
HTTP API	Axum 0.8, Tokio	Fast async server inside the app process
Database	SQLite via SQLx 0.8 (STRICT tables, WAL)	Single-file, zero-admin, crash-safe
Frontend	React 19, TypeScript 6, Vite 8, Tailwind 4	Copied from the web app with minimal changes
PDFs	jsPDF + html2canvas, in the frontend	Invoices, payslips, statutory forms
Passwords / sign-in	Argon2id, JWT (HS256), TOTP 2FA (optional)	Modern hashing; legacy bcrypt rehashed
Secrets	Windows DPAPI	Keys tied to the Windows user, never in plain files
Files	AES-256-GCM	Employee documents and photos encrypted at rest
Money	Integer cents (i64); rust_decimal for rates	No floating point, exact half-up rounding
Spreadsheets	calamine	CSV / Excel / ODS import
Online connectors	reqwest + rustls	Only when a connector is switched on
Installer / updates	NSIS, tauri-plugin-updater (minisign)	Signed updates from GitHub releases

4. Project layout

Folder	Contents
crates/	The Rust libraries (table below)
src-tauri/	Desktop shell, installer settings (tauri.conf.json, tauri.installer.json), icons, MSIX manifest
frontend/	React app; frontend/dist is the built version that ships
migrations/sqlite/	Versioned database migrations (append-only)
tests/	Contract tests against the original app, UI smoke and demo scripts
tools/	release.py, build_msix.py, third_party_notices.py, fixture generators
docs/	Backup, LAN mode, importer, installer and updates, Store, SignPath, user guide
.github/workflows/	release.yml: automatic build, SignPath signing, draft release
claude-plugin/	Claude Code plugin (MCP server + accounting skills)

5. The Rust crates

The crates form a strict dependency order with no cycles: **app-core** → **database** → auth / audit / ledger → operations, employees, payroll, reports → **api-server** → src-tauri.

Crate	Responsibility
app-core	Configuration, errors in the original API's shape, money (cents), time, ids, permission keys, DPAPI secrets
database	SQLite pools (one writer, read-only pool), pragmas, migrations, settings
auth	Passwords, JWT, TOTP, login throttling, users and roles
audit	Activity log and its plain-language summaries
ledger	Chart of accounts, journal posting and reversal, balances, trial balance, lock date
operations	Customers, vendors, invoices, bills, expenses, payments, credit notes, taxes, items and stock, imports
employees	Employee records, salaries, encrypted documents
payroll	Sri Lankan payroll (EPF/ETF/PAYE) and US payroll (federal, FICA, FUTA, SUTA, state tax, 941/940/W-2)
reports	Profit and loss, balance sheet, cash flow, ageing, tax return
documents	Encrypted file storage
backup	Encrypted backups, verification, restore, schedule
integrations	Shopify, WooCommerce, PayPal, Stripe connectors
importer	One-time import from the original PostgreSQL database
mcp	cenango-mcp.exe: Model Context Protocol server for Claude
api-server	Axum router, guards, validation, handlers, static files, the accounting-server binary
src-tauri	The desktop program (accounting-desktop.exe) and the in-app updater

6. Data and database

- **Where:** %LOCALAPPDATA%\TallyCheckData (PCs set up before the rename keep using %LOCALAPPDATA%\CenangoAccounting in place). Separate from the program folder %LOCALAPPDATA%\TallyCheck, so installing, updating or uninstalling never touches the books.
- **Contents:** accounting.db (SQLite), files/ (encrypted documents), secrets/ (DPAPI-protected keys), logs/, pre-upgrade/ (a copy of the database taken before each schema upgrade).
- **Every connection:** foreign_keys=ON, journal_mode=WAL, synchronous=FULL, busy_timeout=5000.
- **Schema:** STRICT tables, column names identical to the original PostgreSQL schema, enums as CHECK constraints, dates as YYYY-MM-DD text, timestamps as ISO-8601 UTC.
- **Money** is stored as integer cents; quantities as hundredths. Rounding is exact half-up.

7. Accounting rules (non-negotiable)

- Every financial change posts a balanced double-entry journal entry; the ledger is the only source of balances.
- Every entry balances exactly to the cent (integer comparison).
- Posted entries are immutable (enforced by database triggers). Corrections are reversals; nothing posted is deleted.
- A document and its journal entry are written in one SQLite transaction; retries cannot post twice.
- A books lock date blocks any posting on or before it, for every kind of document.
- Reports (profit and loss, balance sheet, cash flow, ageing, tax) are derived from ledger lines only.

8. Security

Area	How
Sign-in	Argon2id password hashes; JWT access tokens (8 h); optional TOTP 2FA; 5 failed attempts in 15 min → locked; failed sign-ins audited
Permissions	Role-based keys from the original app; order: authentication (401) → permission (403) → validation (400)
Network	Binds 127.0.0.1 by default; non-loopback Host headers refused (DNS rebinding); LAN only from allowed private networks
Secrets	JWT key, storage key, backup and connector keys protected with Windows DPAPI
Files	AES-256-GCM, fresh random IV per file
Backup / restore / updates	Administrator on the main PC only
Logs	Never contain passwords, hashes, tokens, keys or document contents
Updates	Accepted only if signed with Cenango's update key (minisign); a tampered update is refused
Dependencies	npm audit: 0 vulnerabilities; cargo audit reviewed (remaining findings are not compiled into the Windows app)

9. Main features

- Accounting: chart of accounts, journal entries, bank reconciliation, books lock, multi-currency (foreign documents and bank accounts), revaluation.
- Sales and purchases: customers, vendors, invoices, bills, expenses, payments, credit and debit notes, statements.
- Taxes: user-defined tax codes, groups, withholding, tax return report.
- Inventory: products and services, average cost, stock adjustments, discounts.
- Payroll: Sri Lankan EPF/ETF/PAYE runs and statutory forms; US payroll runs with Form 941/940/W-2 figures.
- Reports: profit and loss (with comparisons), balance sheet, cash flow, ageing, account drill-down, CSV export.
- Integrations: CSV/Excel import, bank transactions, Shopify, WooCommerce, PayPal, Stripe, Claude (MCP).
- Administration: users, roles, activity log, company setup wizard, backup and restore, LAN mode, in-app updates.

10. Testing and quality

Check	What it proves	Result
cargo test --workspace	Unit tests and API tests for every module (money, ledger, payroll, taxes, security, updates…)	235 passed
Contract tests	Identical requests against the original NestJS app (Docker + scratch PostgreSQL) and this server	Up to 360 steps per wave
UI smoke test	Drives the real screens in headless Edge against a fresh database	54 / 54 steps
Payroll differential test	Compares the payroll engine with the original JavaScript on 401 generated lines	Only exact half-cent ties differ (documented)
US payroll	Against all 4,794 values of the IRS 2026 wage-bracket tables	All match
cargo clippy -D warnings, cargo fmt	Code quality and formatting	Clean
npm audit, cargo audit	Known vulnerabilities in libraries	0 in shipped npm packages

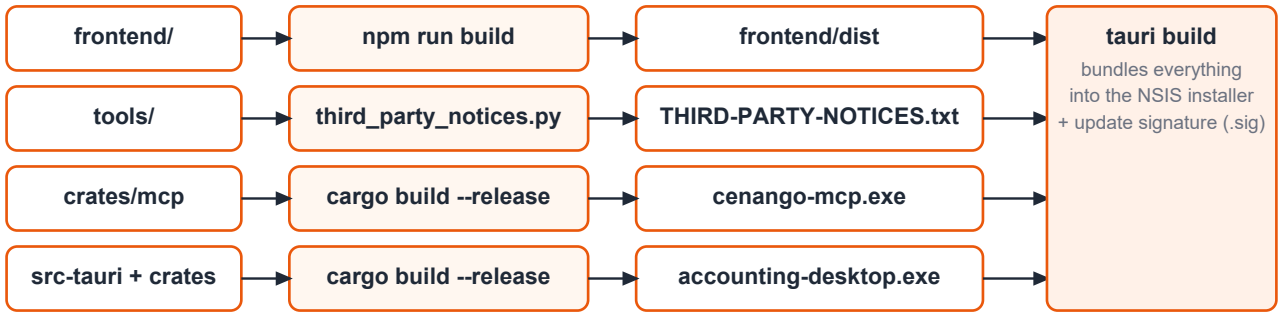
11. How the installer is built

Prerequisites on the build PC: Windows 10/11 with WebView2, Rust 1.99.0 (pinned in rust-toolchain.toml), Node.js, Python 3, and the update signing key in %USERPROFILE%\.tallycheck-signing\ (never inside the project).

```
cd C:\dev\accounting-desktop\frontend
npm ci
npm run build          # 1. React screens  -> frontend\dist

cd C:\dev\accounting-desktop
npm ci                 # 2. pinned Tauri CLI
npm run installer      # 3. notices -> cenango-mcp -> Rust release build -> NSIS installer
```

`npm run installer` runs, in order:



Output	Folder
Installer	target\release\bundle\nsis\TallyCheck_1.0.0_x64-setup.exe
Update signature	…\TallyCheck_1.0.0_x64-setup.exe.sig
Update manifest	…\latest.json (written by tools\release.py)
Program inside	accounting-desktop.exe, cenango-mcp.exe, frontend\, LICENSE-MIT.txt, LICENSE-APACHE.txt, NOTICE.txt, THIRD-PARTY-NOTICES.txt

A full release build takes about 10 minutes. The installer installs per user into %LOCALAPPDATA%\TallyCheck, embeds the WebView2 bootstrapper, and shows the license (NOTICE) page.

12. Releases and in-app updates

```
python tools\release.py --notes "What's new" ^
    --download-base https://github.com/Sammysam/tallycheck/releases/download/v{version}
```

- Raise the version first (Cargo.toml [workspace.package] and src-tauri\tauri.conf.json).
- release.py signs the update with the private update key and writes latest.json (version, notes, date, download address, signature).
- Create a GitHub release v<version> with the installer and latest.json, and publish it.
- Installed copies check https://github.com/Sammysam/tallycheck/releases/latest/download/latest.json when an Administrator opens the app (silently when offline). The status bar shows **Update to x.y.z**; Help → Check for updates checks on request.
- **Update** downloads the installer, verifies its minisign signature against the public key built into the app, closes the database cleanly, runs the installer and reopens TallyCheck. Data is kept, and the database is copied to pre-upgrade\ before any schema change.

Automatic release (GitHub Actions)

.github/workflows/release.yml runs on a v* tag or by hand: builds the installer on a GitHub Windows runner → uploads it to SignPath for code signing (manual approval) → makes the update signature *from the signed file* → writes latest.json → creates a **draft** release for a person to check and publish.

13. Code signing and the SmartScreen warning

Windows shows “Windows protected your PC” for downloaded programs that are not code-signed. **Status (2 November 2026): the installers are not signed yet, so the warning still appears.**

Route	Cost	Status
SignPath Foundation (open source signing)	Free	Workflow, policy page (CODE_SIGNING_POLICY.md) and artifact configuration ready; application to SignPath in progress
Microsoft Security Intelligence submission	Free	Submit each released installer as “Software developer”
Microsoft Store (MSIX, signed by Microsoft)	Free	Built and tested (tools\build_msix.py with the store build switch); not used — distribution is the website download
Azure Artifact Signing / commercial certificate	≈ US$10/month or US$200–600/year	Not chosen
Users: More info → Run anyway, or Unblock, or USB copy	Free	Interim instruction for customers

The warning is considered fixed only after a signed installer, downloaded on a clean PC, opens without it.

14. Licensing

- TallyCheck is licensed **MIT OR Apache-2.0**, at the user's option (LICENSE-MIT, LICENSE-APACHE).
- NOTICE: Copyright (c) 2026 Cenango (Pvt) Ltd; the names “TallyCheck” and “Cenango” and their logos are not licensed.
- THIRD-PARTY-NOTICES.txt lists every library in the program (Rust crates and npm packages) with its license text; it is regenerated before each build by tools\third_party_notices.py.
- New dependencies must allow use in a permissively licensed product (no GPL, AGPL, LGPL-only, SSPL, BUSL or proprietary).
- Privacy policy: PRIVACY.md (data stays on the PC; what goes online, and when).

15. Key design decisions (selection)

Id	Decision
D1	Balances count an entry and its reversal (voids net to zero)
D3, D4	Money as integer cents; exact half-up rounding
D16	2FA optional per user
D17, D20	First-run administrator screen; default chart of accounts on a fresh install
D19	Books lock date enforced on every posting
D25	Any company, any country: company and region are settings; no customer data ships
D26–D30	Tax codes, groups, withholding; zero-decimal and foreign currencies
D37–D41	Import from file, stock, Shopify/WooCommerce/PayPal/Stripe, Claude MCP
D42, D43	US payroll engine, runs, state tax, form figures; accounts chosen per company
D44	Renamed to TallyCheck with the new logo
D45	In-app updates, signed, from GitHub releases
D46	Data folder TallyCheckData; older CenangoAccounting folders used in place
D47	Microsoft Store MSIX build (prepared, not used)
D48	Free code signing via SignPath and GitHub Actions

The full list (D1–D48, with reasons) is in CLAUDE.md §9 in the project.

16. Open items

- Code signing: SignPath approval, then signed releases (SmartScreen).
- Trademark: “Tally” belongs to Tally Solutions (TallyPrime); a name check is advised.
- Confirm the company owns all code (original frontend) and the logo.
- US payroll has not been checked against a real payroll provider; mark it beta.
- LAN mode uses plain HTTP on the office network (documented limitation).